USAC / FACULTAD DE INGENIERÍA

ESTRUCTURAS DE DATOS - x

Día, fecha: xx xx xxxx
Hora de inicio: 17:20
Auxiliar: [por completar]
Semana: [por confirmar]

Escuela de Ingeniería de Ciencias y Sistemas
Facultad de Ingeniería
Universidad de San Carlos de Guatemala

UNIDAD 6

GRAFOS
RECORRIDOS BFS Y DFS

Unidad 6

Tema: Grafos

Definición y características · Operaciones principales
Tipos de Grafos · Recorridos BFS y DFS

Anuncios

xxxx
xxxx
xxxx
xxxx

Completar con los avisos oficiales del curso.

Valor: Valentía, 5 minutos.
Área de conocimiento: 55 minutos.

AGENDA

Inicio de la sesión

Valentía: 5 min.
Definición y características: 10 min.
Operaciones principales: 10 min.

Desarrollo

Tipos de Grafos: 10 min.
Recorridos BFS y DFS: 25 min.

Conocimiento: 55 min.
Sesión completa: 60 min.

Valor de la semana: Valentía

Descripción y relación con la sesión

Actuar correctamente aunque sea difícil o arriesgado.

En la sesión se aplica al preguntar cuando algo no se comprende, reconocer errores en un recorrido y explicar una solución aunque exista temor a equivocarse.

Aplicación del valor

Actividad de 5 minutos:
1 min: explicar el valor.
3 min: proponer un recorrido y revisar con un compañero si respeta las reglas.
1 min: compartir una corrección.

La valentía académica consiste en intentar, pedir ayuda y corregir con honestidad.

Competencias de la planificación

Primer campo visible

«Desarrolla soluciones computacionales basadas en estructuras de datos utilizando estructuras lineales y no lineales junt…»

Segundo campo visible

«Implementa estructuras de datos lineales mediante programación estructurada y orientada a objetos, para gestionar efici…»

Texto visible en la foto: ambos campos aparecen recortados. Completar la redacción desde la planificación original.

2.1 Contenido

No.	Unidad	Tema	Sub Tema
1	6	Grafos	Definición y características
2	6	Grafos	Operaciones principales
3	6	Grafos	Tipos de Grafos
4	6	Grafos	Recorridos BFS y DFS
5			
6			
7			

GRAFOS

Representar conexiones

Unidades relacionadas mediante aristas: de la representación al recorrido

1. Definición formal de grafo

Concepto

Un grafo es una estructura formada por vértices y aristas. Se expresa como G = (V,E), donde V es el conjunto de vértices y E describe sus conexiones.

Los vértices representan entidades y las aristas representan relaciones. El significado depende del problema que se modela.

Ejemplo y análisis

Ejemplos:
Vértices: personas, ciudades o equipos.
Aristas: amistad, rutas o enlaces de red.

Una conexión puede tener dirección o peso. Estas decisiones deben definirse al crear el modelo, antes de implementar sus operaciones.

2. Grafo base de la sesión

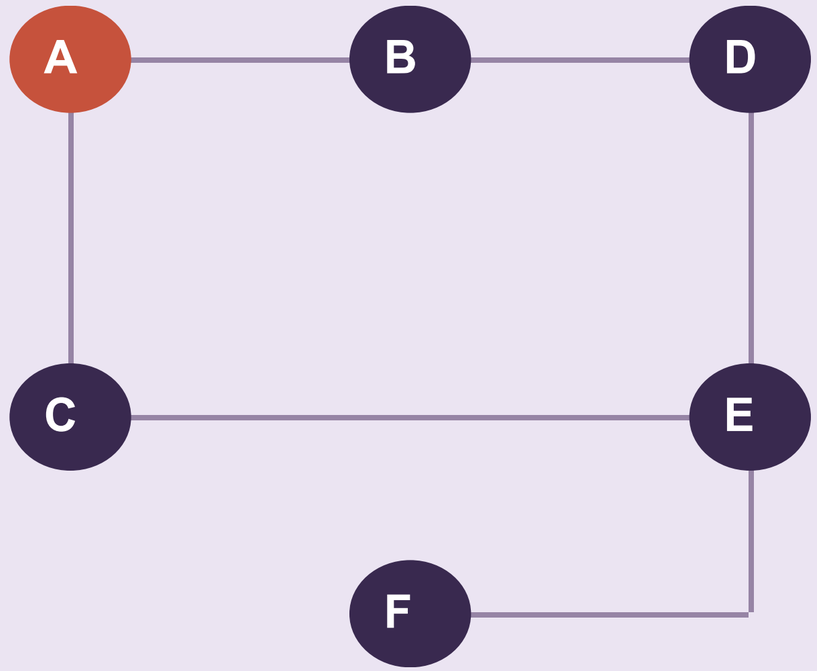
Concepto

Usaremos un grafo no dirigido y sin pesos. Tiene seis vértices y seis aristas.

V = {A,B,C,D,E,F}.
E = {AB,AC,BD,CE,DE,EF}.

En las trazas de BFS y DFS, los vecinos se consideran en orden alfabético. Esto permite comparar los resultados de forma reproducible.

Ejemplo y análisis



6 vértices · 6 aristas · No dirigido

3. Adyacencia e incidencia

Concepto

Dos vértices son adyacentes si existe una arista entre ellos. Una arista es incidente a los vértices que conecta.

No toda pareja de vértices tiene que estar conectada directamente. La ausencia de una arista directa no descarta que exista una ruta a través de otros vértices.

Ejemplo y análisis

En el grafo base:
A y B son adyacentes.
A y E no son adyacentes.
La arista AC es incidente a A y C.

A puede alcanzar E mediante A-C-E.
Una consulta de adyacencia y una consulta de alcanzabilidad resuelven preguntas distintas.

4. Grado de un vértice

Concepto

En un grafo no dirigido simple, el grado es la cantidad de vecinos del vértice. Cada arista contribuye a los grados de sus dos extremos.

La suma de grados es 2|E|. Si se admiten lazos, cada lazo contribuye dos al grado; esa convención debe manejarse de forma explícita.

Ejemplo y análisis

Grados del grafo base:
A: 2; B: 2; C: 2.
D: 2; E: 3; F: 1.

Suma: 12 = 2×6 aristas.

La comprobación detecta omisiones al crear las listas de adyacencia de un grafo no dirigido.

5. Caminos, ciclos y alcanzabilidad

Concepto

Un recorrido por aristas puede conectar varios vértices. Un camino simple no repite vértices y un ciclo simple regresa al origen sin repetir otros vértices.

Un vértice es alcanzable desde otro si existe una secuencia válida de conexiones que permite llegar a él.

Ejemplo y análisis

Camino de A a F:
A-C-E-F, con 3 aristas.

Ciclo del grafo base:
A-B-D-E-C-A.

Si las aristas son dirigidas, cada paso debe respetar su orientación. No se puede recorrer una flecha en sentido contrario sin otra arista que lo permita.

6. Matriz de adyacencia

Concepto

Una matriz de adyacencia reserva una posición para cada pareja de vértices. En un grafo simple sin pesos, 1 indica que existe arista y 0 indica ausencia.

En un grafo no dirigido la matriz es simétrica. Su almacenamiento requiere $O(|V|^2)$ posiciones, aunque existan pocas aristas.

Matriz del grafo base

```
Orden: A B C D E F

A: 0 1 1 0 0 0
B: 1 0 0 1 0 0
C: 1 0 0 0 1 0
D: 0 1 0 0 1 0
E: 0 0 1 1 0 1
F: 0 0 0 0 1 0
```

7. Listas de adyacencia

Concepto

Cada vértice mantiene una colección con sus vecinos. En grafos poco densos, esta representación evita reservar una celda para todas las parejas posibles.

En un grafo no dirigido, cada arista aparece en dos listas. En uno dirigido, puede guardarse únicamente en la lista de su origen.

Ejemplo y análisis

```
Grafo base:
A: [B,C]
B: [A,D]
C: [A,E]
D: [B,E]
E: [C,D,F]
F: [E]

Almacenamiento: O(|V|+|E|). El orden de
los vecinos influye en el orden de
visita.
```

8. Comparar las representaciones

Aspecto	Matriz	Listas	Supuesto
Memoria	$O(\|V\|^2)$	$O(\|V\|+\|E\|)$	Representación convencional
Consultar arista	$O(1)$	$O(\text{grado}(u))$	Lista sin índice
Recorrer vecinos	$O(\|V\|)$	$O(\text{grado}(u))$	Desde un vértice u
Uso frecuente	Grafos densos	Grafos dispersos	Según las operaciones

Las listas con conjuntos o índices adicionales tienen otros costos. Indicar siempre cómo está implementada cada colección.

9. Definir un modelo antes de programar

Concepto

Un modelo debe precisar qué representan sus vértices y conexiones, si hay dirección, pesos, lazos o aristas repetidas.

También necesita identificadores consistentes. La etiqueta visible de un vértice no tiene por qué coincidir con su índice interno en un arreglo.

Ejemplo y análisis

Para el ejemplo:
A→0, B→1, C→2, D→3, E→4, F→5.

Modelo adoptado: simple, no dirigido y sin pesos.

Esta decisión define las reglas para insertar aristas, contar grados y construir las representaciones de la sesión.

10. Insertar un vértice

Concepto

Insertar un vértice agrega una entidad al conjunto V. Debe comprobarse que su identificador no esté registrado y crearse la estructura que almacena sus conexiones.

El costo depende de la representación: una lista necesita una nueva colección; una matriz dinámica puede necesitar ampliar su almacenamiento.

Ejemplo y análisis

Agregar G al grafo base:
V pasa a tener 7 vértices.
G comienza sin vecinos.
El número de aristas sigue siendo 6.

No se crean conexiones automáticamente. Mientras G esté aislado, no será alcanzable desde A.

11. Insertar una arista no dirigida

Concepto

Agregar una arista u-v requiere verificar que ambos extremos existan. En un grafo simple también se comprueba que la arista no esté registrada.

Con listas, se añade v a los vecinos de u y u a los de v. El contador de aristas aumenta una vez, no dos.

Ejemplo y análisis

Agregar B-C al grafo base.
B pasa de [A,D] a [A,C,D].
C pasa de [A,E] a [A,B,E].

|E| pasa de 6 a 7.
Los grados de B y C aumentan uno.

La operación debe completar ambas listas para conservar la simetría.

12. Consultar si existe una conexión

Concepto

La consulta tieneArista(u,v) indica si hay una conexión directa entre los extremos. No debe confundirse con encontrar un camino entre ellos.

En matriz se consulta una celda. En una lista sin índice se busca v entre los vecinos de u.

Ejemplo y análisis

En el grafo base:
tieneArista(A,B) = verdadero.
tieneArista(A,F) = falso.

Sin embargo, F es alcanzable desde A. La ausencia de arista directa no implica ausencia de ruta.

Si un identificador no existe, debe aplicarse una política de error explícita.

13. Obtener y recorrer los vecinos

Concepto

La operación vecinos(u) devuelve o permite recorrer las conexiones que salen de u. En un grafo no dirigido devuelve todos sus vecinos.

Los recorridos BFS y DFS utilizan esta operación repetidamente. Su costo es esencial para analizar el tiempo de esos algoritmos.

Ejemplo y análisis

vecinos(E) = [C,D,F].

Recorrer esa lista inspecciona tres vecinos. Una matriz tendría que revisar toda la fila para encontrar las mismas conexiones.

Si se necesita un orden reproducible, mantener o establecer un criterio para los vecinos.

14. Consultar grados y contadores

Concepto

Las estructuras pueden mantener contadores de vértices, aristas y grados. Esto facilita consultas, pero obliga a actualizarlos correctamente tras cada cambio.

En un grafo dirigido se distinguen grado de entrada y de salida. La longitud de la lista de salidas no indica por sí sola el grado de entrada.

Ejemplo y análisis

En el ejemplo no dirigido:
|V| = 6, |E| = 6.
grado(E) = 3.

Agregar E-A aumenta |E| a 7 y los grados de E y A en uno.

Si solo se cuentan entradas de listas, se obtendrán 14 registros: cada arista aparece dos veces.

15. Eliminar una arista

Concepto

Retirar una arista no dirigida exige eliminar la referencia en ambos extremos. Si la conexión existe, el contador de aristas disminuye una sola vez.

Si la arista no existe, la estructura no debe cambiar de forma accidental. La operación puede devolver un indicador de resultado.

Ejemplo y análisis

Eliminar C-E del grafo base:
C queda con [A].
E queda con [D,F].
|E| pasa de 6 a 5.

Los vértices se conservan. Eliminar una conexión no equivale a eliminar ninguno de sus extremos.

16. Eliminar un vértice y sus conexiones

Concepto

Para retirar un vértice también deben desaparecer todas las aristas incidentes y las referencias que otros vértices mantienen hacia él.

Si se usan índices compactos, puede ser necesario reubicar datos o mantener un mapa actualizado entre etiquetas e índices.

Ejemplo y análisis

Eliminar E del grafo base retira CE, DE y EF.

Quedan 5 vértices y 3 aristas:
AB, AC y BD.
F queda aislado.

Error frecuente: eliminar la lista de E y conservar referencias a E en las listas de C, D o F.

17. Actualizar datos y pesos

Concepto

Un vértice puede tener datos asociados, como nombre o categoría. Una arista puede guardar un peso: distancia, costo o tiempo.

Actualizar un peso no crea un vértice ni una arista nueva. La representación debe distinguir una conexión de peso cero de la ausencia de conexión.

Ejemplo y análisis

Si la conexión A-B tiene tiempo 8 y se actualiza a 5, sus extremos permanecen iguales.

En una matriz ponderada, utilizar cero para «no hay arista» falla cuando cero es un peso válido. Puede emplearse un marcador de ausencia o una estructura de presencia separada.

18. Pseudocódigo de inserción de aristas

Concepto

Este procedimiento corresponde a un grafo simple no dirigido con listas de adyacencia. La política del ejemplo rechaza lazos y evita duplicados.

Comprobar la existencia en una lista puede costar O(grado(u)); no debe describirse toda la operación como constante si incluye esa búsqueda.

Pseudocódigo

```
agregarArista(u,v):
  validar_vertices(u,v)
  si u == v: rechazar
  si v pertenece a adj[u]:
    retornar SIN_CAMBIOS
  adj[u].agregar(v)
  adj[v].agregar(u)
  numeroAristas += 1
  retornar INSERTADA
```

19. Ejercicio de operaciones principales

Concepto

Partir del grafo base y aplicar en orden:
1. Agregar G.
2. Agregar F-G.
3. Eliminar A-B.
4. Consultar los vecinos de F.

Después, revisar los contadores y la simetría de las listas. No se utiliza este grafo modificado en las trazas posteriores.

Ejemplo y análisis

Resultado:
|V| = 7.
|E| = 6: se agregó una arista y se retiró otra.
vecinos(F) = [E,G].
grado(A) = 1; grado(B) = 1.

Toda arista u-v debe aparecer tanto en adj[u] como en adj[v]. Esa comprobación ayuda a detectar cambios incompletos.

20. Grafos no dirigidos

Concepto

Una arista no dirigida conecta dos extremos sin establecer un sentido de recorrido. La relación u-v permite pasar de u a v y de v a u.

Es útil para relaciones simétricas, siempre que esa simetría represente correctamente el problema.

Ejemplo y análisis

El grafo base es no dirigido.
AB representa la misma arista que BA.

Una amistad mutua o un enlace utilizable en ambos sentidos pueden modelarse así. Si el problema permite moverse solo en un sentido, un grafo no dirigido ocultaría esa restricción.

21. Grafos dirigidos

Concepto

Un grafo dirigido utiliza arcos ordenados: u→v no implica v→u. La orientación determina las rutas permitidas.

Cada vértice tiene grado de entrada y grado de salida. Una representación por listas de salida guarda los destinos de cada origen.

Ejemplo y análisis

Arcos: A→B, A→C, C→B.

A tiene salida 2 y entrada 0.
B tiene entrada 2 y salida 0.

B es alcanzable desde A. A no es alcanzable desde B en ese ejemplo. Las flechas deben respetarse durante BFS y DFS.

22. Grafos ponderados y no ponderados

Concepto

Un grafo ponderado asigna valores a sus aristas. Un grafo no ponderado representa solo la existencia de conexiones, o considera que todos los pasos tienen el mismo costo.

La ponderación y la dirección son propiedades independientes: un grafo puede tener ambas o ninguna.

Ejemplo y análisis

Dos rutas pueden diferir:
A-B: 1 arista, costo 20.
A-C-B: 2 aristas, costo total 7.

BFS encuentra una ruta con el menor número de aristas. No garantiza el costo total mínimo cuando los pesos son distintos. No debe ignorarse esa diferencia.

23. Grafos simples y multigrafos

Concepto

Un grafo simple no dirigido no contiene lazos ni aristas paralelas. Un multigrafo permite varias aristas entre una misma pareja.

La aceptación de lazos y su terminología varía según el modelo. Conviene indicar expresamente qué se permite en la implementación.

Ejemplo y análisis

Un lazo conecta A consigo mismo. Dos aristas distintas entre A y B son paralelas.

Varias rutas de transporte entre dos ciudades pueden requerir conservar conexiones independientes. Una colección que elimina duplicados perdería esa información si se usa sin adaptar el modelo.

24. Grafos conexos y desconectados

Concepto

Un grafo no dirigido es conexo si existe un camino entre cualquier pareja de vértices. Si no lo es, se divide en componentes conexas.

Un recorrido desde un origen visita únicamente su componente. Para cubrir todo el grafo deben iniciarse nuevos recorridos desde vértices aún no visitados.

Ejemplo y análisis

El grafo base es conexo.
Si se agrega G sin aristas, queda desconectado: una componente contiene A-F y otra contiene solo G.

En grafos dirigidos, la conectividad necesita distinguir el sentido de las rutas; no basta con aplicar la definición no dirigida.

25. Grafos cíclicos y acíclicos

Concepto

Un grafo cíclico contiene al menos un ciclo. Uno acíclico no contiene ciclos del tipo considerado.

La presencia de un ciclo depende de las aristas y de su orientación. Visitar un vértice ya marcado no implica automáticamente un ciclo en cualquier variante del algoritmo.

Ejemplo y análisis

El grafo base contiene A-B-D-E-C-A.

En un grafo no dirigido, el enlace de regreso al padre no es por sí solo un nuevo ciclo.

En un grafo dirigido, puede utilizarse el estado de la exploración activa para detectar retornos dentro de la rama actual.

26. Árboles y bosques como casos de grafos

Concepto

Un árbol es un grafo no dirigido conexo y sin ciclos. Para $n \geq 1$ tiene $n-1$ aristas. Entre dos vértices existe un único camino simple.

Un bosque es un grafo no dirigido sin ciclos que puede tener varias componentes; cada componente es un árbol.

Ejemplo y análisis

Si retiramos CE del grafo base, quedan 6 vértices y 5 aristas. El grafo sigue conectado y ya no tiene el ciclo: es un árbol.

Esto muestra que un árbol es un caso particular de grafo. No todo grafo tiene una raíz o una única ruta entre sus vértices.

27. Grafos bipartitos

Concepto

Un grafo no dirigido es bipartito si sus vértices pueden separarse en dos conjuntos sin aristas dentro de un mismo conjunto. Todas las conexiones cruzan de un conjunto al otro.

Una aplicación habitual es representar relaciones entre dos categorías de entidades.

Ejemplo y análisis

Ejemplo: estudiantes y proyectos. Aristas: estudiantes asignados a proyectos.

Un grafo no dirigido es bipartito si y solo si no tiene ciclos de longitud impar. El grafo base incluye un ciclo de 5 aristas, por lo que no es bipartito.

28. Objetivo y estado de un recorrido

Concepto

Un recorrido explora vértices y aristas desde un punto de inicio. Utiliza un conjunto de visitados para evitar procesar indefinidamente los ciclos.

Puede registrar padres y otra información útil. Para comparar trazas, se deben fijar el origen y el orden de inspección de los vecinos.

Ejemplo y análisis

Para las trazas volvemos al grafo base:
V = {A,B,C,D,E,F}.
E = {AB,AC,BD,CE,DE,EF}.
Origen: A.
Vecinos: orden alfabético.

BFS y DFS pueden visitar los mismos vértices en diferente orden y producir árboles de padres distintos.

29. BFS: recorrido en anchura

Concepto

BFS explora primero el origen, después sus vecinos y luego los vértices a dos aristas, y así sucesivamente.

Utiliza una cola FIFO: el primer vértice que entra es el primero que se retira para explorar sus vecinos. La marca se coloca al descubrir, antes de encolar.

Ejemplo y análisis

Niveles desde A:
Distancia 0: A.
Distancia 1: B, C.
Distancia 2: D, E.
Distancia 3: F.

Orden de procesamiento con vecinos alfabéticos:
A, B, C, D, E, F.

30. Pseudocódigo de BFS

Concepto

Marcar al encolar evita agregar varias veces el mismo vértice cuando existen diferentes rutas hacia él.

El padre se fija en el primer descubrimiento. Para grafos sin pesos, la distancia del vecino descubierto es la distancia del padre más uno.

Pseudocódigo

```
BFS(G,s):
  marcar(s); distancia[s] = 0
  encolar(s)
  mientras cola no vacia:
    u = desencolar()
    para v en vecinos(u):
      si v no marcado:
        marcar(v); padre[v] = u
        distancia[v] = distancia[u]+1
        encolar(v)
```

31. Traza de la cola de BFS

Paso	Se procesa	Se descubre	Cola al terminar
Inicio	—	A	[A]
1	A	B, C	[B,C]
2	B	D	[C,D]
3	C	E	[D,E]
4	D	Ninguno	[E]
5	E	F	[F]
6	F	Ninguno	[]

D no vuelve a encolar E: E ya fue marcado al descubrirse desde C. Cada vértice entra en la cola una sola vez.

32. Padres y rutas mínimas con BFS

Concepto

En un grafo sin pesos, BFS encuentra la menor cantidad de aristas desde el origen hasta cada vértice alcanzable.

El arreglo de padres permite reconstruir un camino en sentido inverso, desde el destino hasta el origen. Después se invierte la secuencia.

Ejemplo y análisis

Padres desde A:
B←A; C←A; D←B; E←C; F←E.

Para reconstruir A→F:
F←E←C←A.
Invertir: A-C-E-F.

Distancia: 3 aristas. Puede existir más de un camino mínimo; el orden de vecinos decide cuál se registra.

33. DFS: recorrido en profundidad

Concepto

DFS avanza por una rama mientras pueda descubrir nuevos vértices. Cuando no quedan vecinos pendientes, retrocede para continuar otra rama.

Puede implementarse mediante recursión o una pila explícita. El orden de vecinos define cuál de las rutas disponibles se explora primero.

Ejemplo y análisis

Desde A, con vecinos alfabéticos:
A → B → D → E → C.
En C no quedan vecinos nuevos: retroceder hasta E.
Después visitar F.

Orden de descubrimiento:
A, B, D, E, C, F.
DFS no garantiza rutas con menos aristas.

34. Pseudocódigo recursivo de DFS

Concepto

El vértice se marca antes de explorar sus vecinos. Así, si se vuelve a encontrar por otra arista, no se realiza otra llamada recursiva.

Las llamadas pendientes forman una pila. En un grafo profundo, la profundidad de la recursión puede llegar a O(|V|).

Pseudocódigo

```
DFS(u):
  marcar(u)
  procesar(u)
  para v en vecinos(u):
    si v no marcado:
      padre[v] = u
      DFS(v)

Inicio: DFS(A).
Los vecinos se recorren
alfabéticamente.
```

35. Traza de avance y retroceso de DFS

Concepto

Una llamada recursiva termina cuando todos los vecinos del vértice fueron inspeccionados. El retorno no implica descubrir otra vez el padre.

Es útil distinguir el orden de descubrimiento del orden de finalización: describen momentos diferentes del algoritmo.

Ejemplo y análisis

Descubrir: A, B, D, E, C.
Finalizar C y regresar a E.
Descubrir F y finalizarlo.
Finalizar E, D, B y A.

Descubrimiento: A,B,D,E,C,F.
Finalización: C,F,E,D,B,A.

Padres: B←A, D←B, E←D, C←E, F←E.

36. DFS con pila explícita

Concepto

Una pila permite implementar profundidad sin depender de la pila de llamadas del lenguaje.

Para reproducir exactamente la versión recursiva, puede guardarse por cada vértice el siguiente vecino pendiente. Apilar todos los vecinos de golpe puede cambiar el orden o el árbol de padres.

Ejemplo y análisis

Estado de una entrada de pila:
(vértice, siguiente vecino por revisar).

Si hay un vecino nuevo, marcarlo y apilarlo.
Si se agotaron los vecinos, retirar la entrada.

El criterio de marcado debe ser consistente. No se debe afirmar que cualquier versión con pila reproduce la misma traza recursiva.

37. Comparación entre BFS y DFS

Aspecto	BFS	DFS	Condición
Estructura auxiliar	Cola FIFO	Pila / recursión	Exploración
Prioridad	Cercanía por niveles	Profundizar rama	Desde el origen
Ruta con menos aristas	Sí	No garantizada	Grafo sin pesos
Tiempo	O(|V|+|E|)	O(|V|+|E|)	Listas de adyacencia
Memoria auxiliar	O(|V|)	O(|V|)	Sin contar el grafo

Con matriz de adyacencia, recorrer todo el grafo puede costar $O(|V|^2)$. Un solo origen no cubre componentes desconectadas.

38. Recorrer un grafo desconectado

Concepto

Para visitar todos los vértices se recorre la colección completa y se inicia BFS o DFS desde cada vértice que aún no esté marcado.

En un grafo no dirigido, cada nuevo inicio corresponde a una componente diferente. Las marcas deben conservarse entre los recorridos.

Ejemplo y análisis

Agregar G aislado al grafo base.

Un recorrido desde A visita A-F.
G sigue sin marcar.
El recorrido general inicia luego desde G.

Resultado: 2 componentes.

No confundir este conteo con componentes fuertemente conexas de un grafo dirigido.

39. Ejercicio final: comparar y justificar

Concepto

En el grafo base, ejecutar BFS y DFS desde A con vecinos alfabéticos. Registrar el orden y reconstruir un camino hacia F.

Explicar una diferencia entre las dos respuestas. Si se detecta un error, corregirlo y compartir el motivo: esa acción concreta aplica el valor de Valentía.

Ejemplo y análisis

BFS: A,B,C,D,E,F.
Camino por padres: A-C-E-F, 3 aristas.

DFS: A,B,D,E,C,F.
Camino por padres: A-B-D-E-F, 4 aristas.

Ambos alcanzan F. Solo BFS garantiza aquí el menor número de aristas. El camino depende de los padres, no de copiar toda la lista de visita.

Conceptos clave aprendidos

Definición y características
Vértices, aristas y adyacencia
Operaciones principales
Tipos de Grafos

Recorridos BFS y DFS
Visitados, padres y distancias
Cola frente a pila
Valentía para preguntar y corregir

Referencias

Definición, operaciones y recorridos

Robert Sedgewick y Kevin Wayne. Algorithms, 4th edition.

Undirected Graphs.
https://algs4.cs.princeton.edu/41graph/

Dirección y características

Princeton University.
Material de grafos dirigidos.

Directed Graphs.
https://algs4.cs.princeton.edu/42digraph/

Recursos y práctica

Práctica de los recorridos

Repetir las trazas con el grafo base y contrastar la cola de BFS con el avance y retroceso de DFS.

Cambiar el orden de vecinos y verificar qué cambia: el orden de visita o los padres pueden variar; las distancias mínimas de BFS se mantienen.

Cuatro subtemas de la planificación

Alcance de la semana:
1. Definición y características.
2. Operaciones principales.
3. Tipos de Grafos.
4. Recorridos BFS y DFS.

No se incorporan filas adicionales al cuadro de contenido.

PAUSA DE HIDRATACIÓN

DUDAS

¿En qué se distinguen arista y camino?
¿Cuándo usar BFS y cuándo DFS?
¿Por qué marcar los visitados?
¿Cómo aplicamos la valentía al aprender?